A Project on
Database Management System (UCS310)

Submitted By:
Aarush Aggarwal **(102303300)**
Harpuneet Singh **(102483073)**
Piyush Taneja **(102303307)**
Pragati Arora **(102303310)**
2C23

THEATER MANAGEMENT SYSTEM

Submitted To:
Dr. Shashank Singh

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY,
(A DEEMED TO BE UNIVERSITY),
PATIALA, PUNJAB
INDIA

January 2025 - May 2025

Faculty Signature:

Declaration

We, the undersigned, are students of Thapar Institute of Engineering and Technology enrolled in the course Database Management Systems under the guidance of Dr. Shashank Singh. We hereby declare that our group project, titled "Theatre Management System," is an original piece of research work carried out by us. This project has been developed under the Computer Science and Engineering Department as a part of the academic curriculum for Database Management System (UCS310). We have not submitted this project elsewhere for any other academic credit, award, or recognition. We affirm that all data and information provided in this project is authentic to the best of our knowledge. Our group has been systematically assigned to design and implement a database system that efficiently manages the operational and administrative data of a theatre. This system includes modules for ticket booking, seat management, show scheduling, employee management, and financial transactions, among others.

This declaration is to assert that the project is the result of our own intellectual efforts and collective understanding of the subject.

All the sources of information and data used in the preparation of this project have been duly acknowledged. In witness whereof, we have executed this declaration as of the date last written below: DATE: 04-05-2025

Aarush Aggarwal (102303300) – Project Administration, Investigation, Documentation Work, Basic Schema Design, R&D, Ideology Analysis

_____

Harpuneet Singh (102483073) – Project Administration, Sq / PLSql code Implementation, Ideology Analysis, Investigation

_____

Piyush Taneja (102303307) – Sq / PLSql code Implementation, Methodology, Basic Schema Design, Ideology Analysis

_____

Pragati Arora (102303310) – Table Normalisation, Documentation Work, Investigation, Ideology Analysis, Validation

_____

Abstract

This project report presents the design and implementation of a Theatre Management System (TMS) developed by students of Thapar Institute of Engineering and Technology as part of the Database Management Systems Course. The primary objective of this system is to streamline theatre operations by integrating ticket booking, seat management, show scheduling, employee administration, and financial transactions into a unified database framework.

The development of the TMDMS was initiated to address the complexities and inefficiencies inherent in traditional theatre management, characterised by manual ticketing processes and disjointed data storage. Utilising Live SQL as the database technology, the system offers robust data handling capabilities, ensuring data integrity, security, and accessibility.

The methodology adopted for this project involved a comprehensive analysis of theatre workflows, identification of key data entities and relationships, followed by database normalisation to minimise redundancy. User-friendly interfaces were created to facilitate seamless interactions by theatre staff and administrators with the database. The system was tested in a simulated environment to validate functionality, efficiency, and error-handling capabilities.

Key features of the TMDMS include automated ticketing and seat allocation, real-time show scheduling and notifications, efficient resource management for theatre staff, and integrated financial management modules. The project outcomes indicate a significant improvement in operational efficiency, with potential implications for enhanced customer experience and revenue management.

This report details each phase of the project from conceptualisation through implementation, including challenges encountered, solutions implemented, and insights gained. The TMDMS represents a scalable and efficient solution that could be adapted to various theatre settings, promising enhanced operational effectiveness and customer satisfaction.

INDEX

I. INTRODUCTION, PROBLEM STATEMENT & OBJECTIVES

The Theatre Management System seeks to optimise and automate theatre operations, encompassing ticket booking, show scheduling, transaction tracking, and customer information management. The goal is to develop a resilient database system that guarantees efficient data management and delivers a smooth experience for theatre administrators and patrons. In the modern digital era, the entertainment sector is swiftly transforming, with theatres progressively using online platforms for ticket sales and management. Our objective as engineers is to create a complete database management system specifically designed to assist theatre owners in transitioning and optimising their operations. This system would encompass essential components for an online theatre platform, such as ticket buying, seat allocation, show schedules, customer profiles, payment processing, and analytics, improving efficiency and experience for both theatre operators and consumers.

1. Problem Statement:

Manual management of theatre operations is time-intensive and susceptible to mistakes, resulting in ticket reservations, financial transactions, and performance schedule inefficiencies. Prevalent problems encompass multiple bookings from insufficient real-time synchronisation, erroneous transaction records that hinder revenue tracking, and ineffective scheduling that leads to underused screens or conflicting showtimes. These inefficiencies impact both client pleasure and the theatre's revenue.

The present theatre management environment is disjointed, with numerous institutions continuing to depend on antiquated technologies, such as paper logs or legacy software that lacks contemporary functionalities like automation and real-time updates. These systems are inefficient and challenging to maintain, heightening the risk of data loss and operational impediments. Furthermore, personnel frequently require comprehensive training to operate these antiquated interfaces, resulting in prolonged processing times and consumer discontent.

This project seeks a contemporary, database-driven Theatre Management System that guarantees precision, scalability, and user-friendliness. The suggested system would optimise operations by integrating automated ticket booking, real-time seat availability updates, secure transaction processing, and an easy manager scheduling module. The intuitive interface enables cashiers to purchase tickets swiftly without requiring technical skills, while management can easily add or edit movies, showtimes, and pricing structures. Implementing this solution enables theatres to augment operating efficiency, minimise mistakes, and elevate the overall client experience.

2. Scope:

This project aims to provide a ticket booking kiosk for cashiers and a management interface for theatre managers. The system is engineered to enable rapid and frictionless ticket booking while providing real-time information on seat availability and movie showtimes. It primarily functions as an efficient solution for the everyday operations of a theatre rather than a thorough long-term data storage system.

The system is designed to manage essential theatre operations, including movie management, scheduling screenings, and processing ticket reservations. It offers an intuitive interface for cashiers to handle ticket sales efficiently and for management to add, edit, or eliminate movies and showtimes as required. The initiative does not encompass online ticket bookings, mobile applications, or the provision of concessions, including food and beverages. Furthermore, previous records of transactions or audience attendance are retained just as long as operationally required, hence maintaining the system's efficiency and optimisation for daily usage.

This project intends to provide an effective, user-friendly system for the theatre's in-person booking procedure, enhancing efficiency without superfluous complexity. The omission of online ticketing and concession management guarantees that the technology is tailored and refined for its Designated users—cashiers managing ticket sales and managers supervising scheduling and theatrical operations.

3. Methodology & Technology Used:

The development of the Theatre Management System follows a structured methodology, incorporating database-driven design and modular development. The system is built using Live Oracle for database management, ensuring efficient storage and retrieval of movie schedules, bookings, and transaction records. Normalisation techniques are applied to minimise redundancy and maintain data integrity.

A. *Entity-Relationship Model Design:*

Entity-relationship (ER) modelling defines relationships between entities such as customers, shows, movies, seats, transactions, etc. The system is structured around key entities such as Movies, Shows, Halls, Customers, Booked Tickets, Transactions, Seats, and Pricing. Normalisation techniques ensure minimal redundancy and maintain data integrity. Cardinality constraints are applied to define relationships (e.g., a

show can be booked multiple times, but each booked ticket is linked to a single transaction).

B. ***Database Implementation:***

Live Oracle-based SQL is used as the Relational Database Management System (RDBMS) for structured data storage. Foreign key constraints enforce referential integrity among tables (e.g., Booked_Ticket is linked to Customer and Transaction).

4. Expected Outcome:

The successful implementation of the Theatre Management System will result in the following key outcomes:

A. Efficient Ticket Booking Process

- Cashiers can quickly book tickets through a user-friendly interface.
- Real-time seat availability prevents double bookings and ensures accurate seat allocation.

B. Seamless Show and Movie Management

- Managers can easily add, modify, and remove movie listings and show schedules.
- Automated scheduling prevents overlapping or duplicate showtimes in the same hall.

C. Secure and Accurate Transactions

- Each booked ticket is linked to a unique transaction ID, ensuring traceability.
- Integration of price mapping based on seat category and hall type optimises revenue management.

D. Data Integrity and Reduced Errors

- The use of relational database constraints ensures referential integrity between customers, bookings, and transactions.
- Stored procedures and triggers automate updates, reducing manual intervention and human errors.

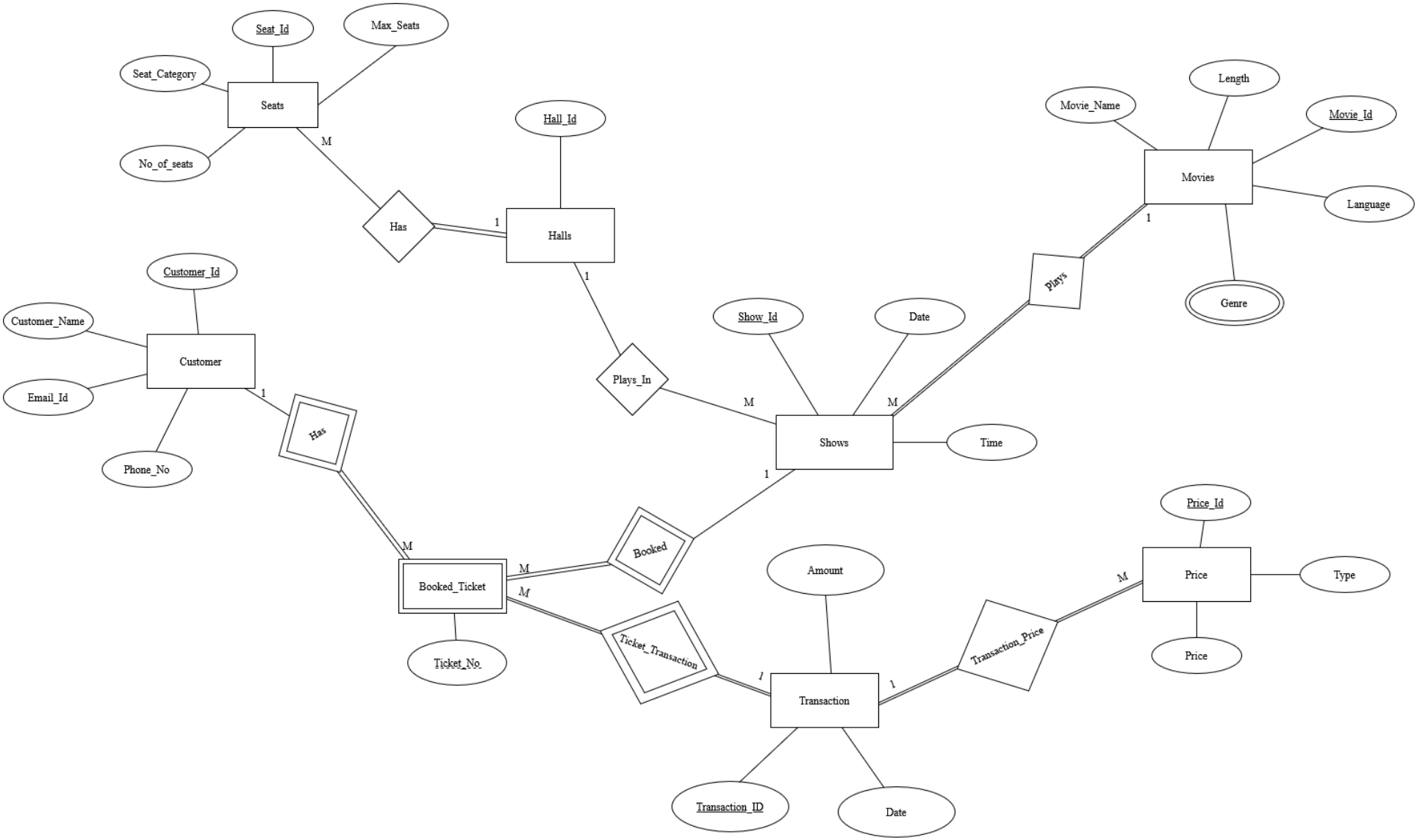
E. Role-Based Access Control

- Cashiers can access ticket booking functionalities, while managers can handle movies, show schedules, and pricing.
- Authentication mechanisms prevent unauthorised modifications to the database.

F. Improved Operational Speed and Scalability

- The system is designed to handle multiple bookings simultaneously, ensuring quick processing even during peak hours.
- Scalability options allow future integration of online booking and mobile applications.

II. Entity-Relationship Diagram



III. Entity-Relationship Tables

Below is the structure of the tables, including **Primary Keys (PK)**, **Foreign Keys (FK)**, and additional **constraints** to ensure data integrity.

1. Customer Table

Column Name	Data Type	Constraints
Customer_ID	NUMBER	PRIMARY KEY, NOT NULL
Customer_Name	VARCHAR(50)	NOT NULL
Email_ID	VARCHAR(100)	UNIQUE, NOT NULL
Phone_No	VARCHAR(15)	UNIQUE, NOT NULL

Explanation:

- Customer_ID is the **Primary Key (PK)** to identify each customer uniquely.
- Email_ID and Phone_No are marked as **UNIQUE** to avoid duplicate entries.

2. Seats Table

Column Name	Data Type	Constraints
Seat_ID	NUMBER	PRIMARY KEY, NOT NULL
Seat_Category	VARCHAR(20)	NOT NULL
No_of_Seats	NUMBER	CHECK (No_of_Seats > 0), NOT NULL
Max_seats	NUMBER	NOT NULL
Hall_ID	NUMBER	FOREIGN KEY REFERENCES Halls (Hall_ID), NOT NULL

Explanation:

- Seat_ID is the **Primary Key (PK)**.
- Hall_ID is the Foreign Key referencing the Halls Table.
- The No_of_Seats column has a **CHECK constraint** to ensure it is greater than zero.
- The Class column has a **CHECK constraint** to restrict values to predefined categories.

3. Halls Table

Column Name	Data Type	Constraints
Hall_ID	NUMBER	PRIMARY KEY, NOT NULL

Explanation:

- Hall_ID is the **Primary Key (PK)**.

4. Movies Table

Column Name	Data Type	Constraints
Movie_ID	NUMBER	PRIMARY KEY, NOT NULL
Movie_Name	VARCHAR(100)	NOT NULL
Length	NUMBER	CHECK (Length > 0), NOT NULL
Language	VARCHAR(50)	NOT NULL
Genre	VARCHAR(50)	CHECK (Genre IN ('Action', 'Drama', 'Comedy', 'Horror'))

Explanation:

- Movie_ID is the **Primary Key (PK)**.
- The Length column has a **CHECK constraint** to ensure it is positive.
- The Genre column has a **CHECK constraint** for predefined genres.

5. Shows Table

Column Name	Data Type	Constraints
Show_ID	NUMBER	PRIMARY KEY, NOT NULL
Date	DATE	NOT NULL
Time	TIME	NOT NULL
Movie_ID	NUMBER	FOREIGN KEY REFERENCES Movies(Movie_ID), NOT NULL
Hall_ID	NUMBER	FOREIGN KEY REFERENCES Halls(Hall_ID), NOT NULL

Explanation:

- Show_ID is the **Primary Key (PK)**.
- Movie_ID and Hall_ID are **Foreign Keys (FK)** referencing their respective tables.

6. Booked_Ticket Table

Column Name	Data Type	Constraints
Ticket_No	NUMBER	PRIMARY KEY, NOT NULL
Transaction_ID	NUMBER	FOREIGN KEY REFERENCES Seats(Seat_ID), NOT NULL
Show_ID	NUMBER	FOREIGN KEY REFERENCES Shows(Show_ID), NOT NULL
Seat_ID	NUMBER	FOREIGN KEY REFERENCES Seat(Seat_ID), NOT NULL
Hall_ID	NUMBER	FOREIGN KEY REFERENCES Hall(Hall_ID), NOT NULL
Customer_ID	NUMBER	FOREIGN KEY REFERENCES Customers(Customer_ID), NOT NULL

Explanation:

- Ticket_No is the **Primary Key (PK)**.
- Transaction_ID, Show_ID, Seat_ID, Hall_ID and Customer_ID are **Foreign Keys (FK)** referencing their respective tables.

7. Price Table

Column Name	Data Type	Constraints
Price_ID	NUMBER	PRIMARY KEY, NOT NULL
Type	VARCHAR(20)	CHECK (Type IN ('VIP', 'Regular')), NOT NULL
Price	DECIMAL(10,2)	CHECK (Price > 0), NOT NULL

Explanation:

- Price_ID is the **Primary Key (PK)**.
- The Type column has a **CHECK constraint** for predefined categories.
- The Price column has a **CHECK constraint** to ensure it is positive.

8. Transaction Table

Column Name	Data Type	Constraints
Transaction_ID	NUMBER	PRIMARY KEY, NOT NULL
Date	DATE	DEFAULT CURRENT_DATE, NOT NULL
Amount	DECIMAL(10,2)	CHECK (Amount > 0), NOT NULL
Price_ID	NUMBER	FOREIGN KEY REFERENCES Price(Price_ID)

Explanation:

- Transaction_ID is the **Primary Key (PK)**.
- Price_ID is a **Foreign Key (FK)** referencing the Price table.
- The Date column has a default value of the current date.

IV. Summary of Primary Keys, Foreign Keys & Relationships

1. Primary Keys

- Each table has a unique identifier as its primary key:
  - Customer: Customer_ID
  - Seats: Seat_ID
  - Halls: Hall_ID
  - Movies: Movie_ID
  - Shows: Show_ID
  - Booked_Ticket: Ticket_No
  - Price: Price_ID
  - Transaction: Transaction_ID

2. Foreign Keys

- Relationships between tables are maintained using foreign keys:
  - Shows → Movies (Movie_ID)
  - Shows → Halls (Hall_ID)
  - Seats → Halls(Hall_ID)
  - Booked_Ticket → Seats (Seat_ID)
  - Booked_Ticket → Transaction (Transaction_ID)
  - Booked_Ticket →Halls (Hall_ID)
  - Booked_Ticket → Customers (Customer_ID)
  - Booked_Ticket → Shows (Show_ID)
  - Transaction → Price (Price_ID)

3. Added Constraints

1. **NOT NULL** constraints on all critical columns to prevent missing data.
2. **UNIQUE constraints** on columns like Email_ID, Phone_No, and foreign key relationships where necessary.
3. **CHECK constraints** for data validation on numeric fields (Price, Length, etc.) and categorical fields (Class, Genre, etc.).
4. Default values for certain columns like transaction date (Date) in the Transaction table.

V. Description of Tables

1. Customers

a. Customer_id (Primary Key): A unique numeric identifier for each customer. Serves as the primary key of the table.

b. Customer_name: The full name of the customer. Cannot be NULL.

c. Email_id: The customer's email address for contact and booking information. Cannot be NULL.
d. Phone_no: The customer's 10-digit phone number, used for contacts. It can contain null values, indicating that the phone number may not be available for all customers.

Table Creation:

```
CREATE TABLE Customers (
    Customer_id   NUMBER(10) PRIMARY KEY,
    Customer_name VARCHAR2(50) NOT NULL,
    Email_id      VARCHAR2(50) NOT NULL,
    Phone_no      NUMBER(10)
)
```

```
Table created.
```

Insertion:

```
INSERT INTO Customers VALUES (1,'John Doe','john.doe@gmail.com',1234567890)
```

```
1 row(s) inserted.
```

Statement 6

```
INSERT INTO Customers VALUES (2,'Jain Smith','Jain.Smith@gmail.com',9574637282)
```

```
1 row(s) inserted.
```

Statement 7

```
INSERT INTO Customers VALUES (3,'Michael Johnson','Michael.Johnson@gmail.com',8464632828)
```

```
1 row(s) inserted.
```

Table Display:

```
SELECT * FROM Customers
```

CUSTOMER_ID	CUSTOMER_NAME	EMAIL_ID	PHONE_NO
1	John Doe	john.doe@gmail.com	1234567890
2	Jain Smith	Jain.Smith@gmail.com	9574637282
3	Michael Johnson	Michael.Johnson@gmail.com	8464632828
4	Emily Davis	Emily.Davis@gmail.com	7836483738
5	Robert Brown	Robert.Brown@gmail.com	6483748393
6	Daniel Wilson	Daniel.Wilson@gmail.com	9123456789
7	Sophia Martinez	Sophia.Martinez@gmail.com	9988776655
8	William Anderson	William.Anderson@gmail.com	8877665544
9	Olivia Taylor	Olivia.Taylor@gmail.com	7766554433
10	James Thomas	James.Thomas@gmail.com	6655443322
11	Ava Moore	Ava.Moore@gmail.com	5544332211
12	Benjamin Jackson	Benjamin.Jackson@gmail.com	4433221100
13	Isabella White	Isabella.White@gmail.com	3322110099

2. Movies

a. Movie_id (Primary Key): Numeric data type with a maximum precision of 10 digits. It uniquely identifies each movie.Serves as the primary key.
b. Movie_name: Stores up to 50 characters for the movie title. Must not be NULL.
c. Length: Stores a numeric value (up to 10 digits) representing the movie's duration in minutes.
d. Language: Stores up to 20 characters for the language in which the movie is primarily made.

Table Creation:

```
CREATE TABLE Movies (
    Movie_id    NUMBER(10) PRIMARY KEY,
    Movie_name  VARCHAR2(50) NOT NULL,
    Length      NUMBER(10),
    Language    VARCHAR2(20)
)

Table created.
```

Insertion:

```
INSERT INTO Movies VALUES (1, 'Movie_1', 120, 'English')

1 row(s) inserted.
```

\

Table Display:

```
SELECT * FROM Movies
```

MOVIE_ID	MOVIE_NAME	LENGTH	LANGUAGE
1	Movie_1	120	English
2	Movie_2	110	Spanish
3	Movie_3	130	French
4	Movie_4	100	German
5	Movie_5	150	Japanese

3. Genre

a. Movie_id: A foreign key referencing the Movie_id in the Movies table. Identifies the movie.
b. Genre: The category of the type of the movie (up to 10 characters).
c. Primary Key (Movie_id, Genre): Composite primary key consisting of "Movie_id" and "Genre" columns.This means a movie can have multiple genres, but a specific genre cannot repeat for the same movie.

Table Creation:

```
CREATE TABLE Genre (
    Movie_id NUMBER(10) REFERENCES Movies(Movie_id),
    Genre    VARCHAR2(10),
    PRIMARY KEY (Movie_id, Genre)
)
```

```
Table created.
```

Insertion:

```
INSERT INTO Genre VALUES (1, 'Action')
```

```
1 row(s) inserted.
```

Table Display:

```
SELECT * FROM Genre
```

MOVIE_ID	GENRE
1	Action
2	Drama
3	Comedy
4	Romance
5	Thriller

4. Price

a. Price_ID(Primary key): A unique identifier (up to 10 digits) for each price record. Acts as the primary key.

b. Type: Describes the seat category/type. Supports names up to 20 characters.

c. Price: The numeric cost (up to 10 digits) for the given seat type.

Table Creation:

```
CREATE TABLE Price (
    Price_id NUMBER(10) PRIMARY KEY,
    Type     VARCHAR2(20),
    Price    NUMBER(10)
)
```

Insertion:

```
INSERT INTO Price VALUES (1, 'Standard', 20)
```

```
1 row(s) inserted.
```

```
INSERT INTO Price VALUES (2, 'Premium',  15)
```

```
1 row(s) inserted.
```

```
INSERT INTO Price VALUES (3, 'VIP',      20)
```

```
1 row(s) inserted.
```

Table Display:

```
SELECT * FROM Price
```

PRICE_ID	TYPE	PRICE
1	Standard	20
2	Premium	15
3	VIP	20
4	Balcony	25
5	Box	30

5. **Halls**

a. Hall_ID: A unique identifier (up to 10 digits) for each hall. Acts as the primary key. Represents a physical movie hall or auditorium in a multiplex/theater.

Table Creation:

```
CREATE TABLE Halls (
    Hall_id NUMBER(10) PRIMARY KEY
)
```

Insertion:

```
INSERT INTO Halls VALUES (1)
```

```
1 row(s) inserted.
```

```
INSERT INTO Halls VALUES (2)
```

```
1 row(s) inserted.
```

```
INSERT INTO Halls VALUES (3)
```

```
1 row(s) inserted.
```

Table Display:

```
SELECT * FROM Halls
```

HALL_ID
1
2
3
4
5

6. **Seats**
    a. Seats_ID (Primary Key): Unique identifier for a seat group/category in a specific hall. Not necessarily one physical seat, often a block of similar seats (e.g., VIP section, row A).
    b. No_of_seats: The current available number of seats in a particular seat category or group. This decreases as tickets are booked and increases on cancellations.
    c. Seat_Category: The type of seat (e.g., 'VIP', 'Regular', 'Balcony'). This is used to calculate pricing and comfort level in the actual venue.
    d. Hall_ID: Identifies which hall or auditorium the seat group belongs to. Ties the seat to a specific show venue.
    e. Max_Seats: The total number of seats originally available in a particular category. Used to reset or recalculate No_of_seats as needed.

Table Creation:

```
CREATE TABLE Seats (
    Seat_id       NUMBER(10) PRIMARY KEY,
    No_of_seats   NUMBER(10),
    Seat_Category VARCHAR2(10),
    Hall_id       NUMBER REFERENCES Halls(Hall_id),
    Max_seats     NUMBER
)
```

```
Table created.
```

Insertion:

```
INSERT INTO Seats VALUES (1,  100, 'Standard', 1, 100)
```

```
1 row(s) inserted.
```

```
INSERT INTO Seats VALUES (2,   20, 'Premium',  1, 20)
```

```
1 row(s) inserted.
```

```
INSERT INTO Seats VALUES (3,   50, 'VIP',      1, 50)
```

```
1 row(s) inserted.
```

```
INSERT INTO Seats VALUES (4,   30, 'Balcony',  1, 30)
```

```
1 row(s) inserted.
```

Table Display:

```
SELECT * FROM Seats
```

SEAT_ID	NO_OF_SEATS	SEAT_CATEGORY	HALL_ID	MAX_SEATS
1	100	Standard	1	100
2	20	Premium	1	20
3	50	VIP	1	50
4	30	Balcony	1	30
5	10	Box	1	10
6	70	Standard	2	70
7	20	Premium	2	20
8	15	VIP	2	15
9	10	Box	2	10
10	25	Balcony	2	25
11	90	Standard	3	90
12	40	Premium	3	40
13	20	VIP	3	20
14	10	Box	3	10
15	15	Balcony	3	15
16	60	Standard	4	60
17	20	Premium	4	20
18	10	VIP	4	10
19	5	Box	4	5
20	8	Balcony	4	8
21	95	Standard	5	95
22	45	Premium	5	45
23	25	VIP	5	25
24	15	Box	5	15
25	10	Balcony	5	10

7. **Shows**
    a. Show_id(Primary key):
    b. Show_date: Date when the show is scheduled.
    c. Time: Time of the show, stored as a string (up to 10 characters).
    d. Hall_id: Foreign key referencing Halls(Hall_id). Links the show to the physical hall where it is being screened.
    e. Movie_id: Foreign key referencing Movies(Movie_id). Associates the show with the movie being played.

Table Creation:

```
CREATE TABLE Shows (
    Show_id   NUMBER(10) PRIMARY KEY,
    Show_Date DATE,
    Time      VARCHAR2(10),
    Hall_id   NUMBER(10) REFERENCES Halls(Hall_id),
    Movie_id  NUMBER(10) REFERENCES Movies(Movie_id)
)

Table created.
```

Insertion:

```
INSERT INTO Shows VALUES (1, TO_DATE('2025-05-05', 'YYYY-MM-DD'), '18:00', 1, 1)

1 row(s) inserted.
```

```
INSERT INTO Shows VALUES (2, TO_DATE('2025-05-05', 'YYYY-MM-DD'), '20:00', 2, 2)

1 row(s) inserted.
```

```
INSERT INTO Shows VALUES (3, TO_DATE('2025-05-06', 'YYYY-MM-DD'), '19:00', 3, 3)

1 row(s) inserted.
```

Table Display:

```
SELECT * FROM Shows
```

SHOW_ID	SHOW_DATE	TIME	HALL_ID	MOVIE_ID
1	05-MAY-25	18:00	1	1
2	05-MAY-25	20:00	2	2
3	06-MAY-25	19:00	3	3
4	06-MAY-25	21:00	4	4
5	07-MAY-25	17:00	5	5

8. **Transactions**
    a. Transaction_id(Primary key): Unique identifier for each transaction. Acts as the primary key.
    b. Amount: Total amount paid in the transaction.
    c. T_date: Date of the transaction.
    d. Price_id: Foreign key referencing Price(Price_id)- indicates ticket type.

Table Creation:

```
CREATE TABLE Transaction (
    Transaction_id NUMBER(10) PRIMARY KEY,
    Amount         NUMBER(10),
    T_Date         DATE,
    Price_id       NUMBER REFERENCES Price(Price_id)
)
```

```
Table created.
```

Insertion:

```
INSERT INTO Transaction VALUES (1,  20, TO_DATE('2025-05-05', 'YYYY-MM-DD'), 1)
```

```
1 row(s) inserted.
```

```
INSERT INTO Transaction VALUES (2,  45, TO_DATE('2025-05-05', 'YYYY-MM-DD'), 2)
```

```
1 row(s) inserted.
```

```
INSERT INTO Transaction VALUES (3,  60, TO_DATE('2025-05-05', 'YYYY-MM-DD'), 3)
```

```
1 row(s) inserted.
```

Table Display:

```
SELECT * FROM Transaction
```

TRANSACTION_ID	AMOUNT	T_DATE	PRICE_ID
1	20	05-MAY-25	1
2	45	05-MAY-25	2
3	60	05-MAY-25	3
4	50	06-MAY-25	4
5	120	07-MAY-25	5

9. Booked Ticket

a. Ticket_no(Primary key): Unique identifier for each booked ticket.
b. Transaction_id: Foreign key referencing Transaction(Trasaction_id). Connects this ticket to a recorded purchase/payment.
c. Show_id: Foreign key referencing Shows(show_id). Indicates which shows this ticket is for.
d. Seat_id: Foreign key referencing Seat(seat_id). Identifies the specific seat reserved.

e. Hall_id: Foreign key referencing Halls(Hall_id). Indicates the physical hall where the show will occur.
f. Customer_id: Foreign key referencing Customers(Customer_id). Identifies the person who booked the ticket.

Table Creation:

```
CREATE TABLE Booked_Ticket (
    Ticket_no     NUMBER(10) PRIMARY KEY,
    Transaction_id NUMBER REFERENCES Transaction(Transaction_id),
    Show_id       NUMBER REFERENCES Shows(Show_id),
    Seat_id       NUMBER REFERENCES Seats(Seat_id),
    Hall_id       NUMBER REFERENCES Halls(Hall_id),
    Customer_id   NUMBER REFERENCES Customers(Customer_id)
)

Table created.
```

Insertion:

```
INSERT INTO Booked_Ticket VALUES (1,  1, 1,  1, 1,  1)

1 row(s) inserted.
```

```
INSERT INTO Booked_Ticket VALUES (2,  2, 1,  2, 1,  2)

1 row(s) inserted.
```

Table Display:

```
SELECT * FROM Booked_Ticket
```

TICKET_NO	TRANSACTION_ID	SHOW_ID	SEAT_ID	HALL_ID	CUSTOMER_ID
1	1	1	1	1	1
2	2	1	2	1	2
3	2	1	2	1	3
4	2	2	7	2	4
5	3	1	3	1	5
6	3	2	8	2	6
7	3	3	13	3	7
8	4	1	4	1	8
9	4	2	10	2	9
10	5	3	14	3	10
11	5	1	5	1	11
12	5	2	9	2	12
13	5	4	19	4	13

VI. Normalisation

Functional Dependencies of the Following Tables:

1. **Seats Table Dependencies:**
   a) Seat_id → Seat_Category
   b) Seat_id → No_of_seats
   c) Seat_id → Class
   d) Seat_id → Max_seats

2. **Halls Table Dependencies:**
   a) Hall_id → Seat_ID
   b) Hall_id → Seats (foreign key referencing Seats table)

3. **Customers Table Dependencies:**
   a) Customer_id → Customer_Name

b) Customer_id → Email_ID
c) Customer_id → Phone_No

4. **Movies Table Dependencies:**
   a) Movie_id → Movie_Name
   b) Movie_id → Length
   c) Movie_id → Language

5. **Genre Table Dependencies:**
   a) (Movie_id, Genre) → Movie_ID
   b) (Movie_id, Genre) → Genre

6. **Shows Table Dependencies:**
   a) Show_id → DDate
   b) Show_id → Time
   c) Hall_id → Halls (foreign key referencing Halls table)
   d) Movie_id → Movies (foreign key referencing Movies table)

7. **Prices Table Dependencies:**
   a) Price_id → Type
   b) Price_id → Price

8. **Booked_Tickets Table Dependencies:**
   a) Ticket_No → Transaction_ID
   b) Ticket_No → Show_ID
   c) Ticket_No → Customer_ID

9. **Transaction Table Dependencies:**
   a) Transaction_ID → Amount
   b) Transaction_ID → T_Date
   c) Price_ID → Price (foreign key referencing Prices table)

Breakdown of Normalisation Process for Each Table

1. **Table 1: Seats (Already Normalised)**
   a) Already in 1st Normal Form (1nf): No repeating groups, all atomic values.
   b) Already in 2nd Normal Form (2nf): No partial dependencies.
   c) Already in 3rd Normal Form (3nf): No transitive dependencies.

2. **Table 2: Halls (Already Normalised)**
   a) 1st Normal Form (1nf): Already satisfied.
   b) 2nd Normal Form (2nf): Already satisfied.
   c) 3rd Normal Form (3nf): Hall_ID determines Seat_ID, which references Seats, so no transitive dependency. Already in 3nf.

3. **Table 3: Customers (Already Normalised)**
   a) Already in 1nf: No repeating groups, all atomic values.
   b) Already in 2nf: No partial dependencies.
   c) Already in 3nf: No transitive dependencies.

4. **Table 4: Movies (Already Normalised)**
   a) Already in 1nf: No repeating groups, all atomic values.
   b) Already in 2nf: No partial dependencies.
   c) Already in 3nf: No transitive dependencies.

5. **Table 5: Genre (Already Normalised)**
   a) Already in 1nf: No repeating groups, all atomic values.
   b) Already in 2nf: No partial dependencies.
   c) Already in 3nf: No transitive dependencies.

6. **Table 6: Shows (Already Normalised)**
   a) 1nf: No repeating groups, all atomic values.
   b) 2nf: No partial dependencies.
   c) 3nf: DDate and Time depend only on Show_ID, already in 3NF.

7. **Table 7: Prices (Already Normalised)**
   a) Already in 1nf: No repeating groups, all atomic values.
   b) Already in 2nf: No partial dependencies.
   c) Already in 3nf: No transitive dependencies.

8. **Table 8: Booked_Tickets (Already Normalised)**
   a) Already in 1nf: No repeating groups, all atomic values.
   b) Already in 2nf: No partial dependencies.
   c) Already in 3nf: No transitive dependencies.

9. **Table 9: Transaction (Already Normalised)**
   a) Already in 1nf: No repeating groups, all atomic values.
   b) Already in 2nf: No partial dependencies.
   c) Already in 3nf: No transitive dependencies.

VII. Database Implementation

1. Creating tables:

```
-- Table to store customer details
CREATE TABLE Customers (
    customer_id NUMBER(10) PRIMARY KEY,
    Customer_name VARCHAR2(50) NOT NULL,
    Email_id VARCHAR2(50) NOT NULL,
    Phone_no NUMBER(10)
);

-- Table to store movie details
CREATE TABLE Movies (
    movie_id NUMBER(10) PRIMARY KEY,
    Movie_name VARCHAR2(50) NOT NULL,
    length NUMBER(10),
    language VARCHAR2(20)
);

-- Table to associate genres with movies (many-to-many-like structure)
CREATE TABLE Genre (
```

```
    Movie_id NUMBER(10) REFERENCES Movies(Movie_id),
    Genre VARCHAR2(10),
    PRIMARY KEY (Movie_id, Genre)
);

-- Table to store hall information
CREATE TABLE Halls (
    hall_id NUMBER(10) PRIMARY KEY
);

-- Table to store seat details within halls
CREATE TABLE Seats (
    Seat_id NUMBER(10) PRIMARY KEY,
    No_of_seats NUMBER(10),
    Seat_Category VARCHAR2(10),
    Hall_id NUMBER(10) REFERENCES Halls(Hall_id)
);

-- Table to define pricing for each seat type
CREATE TABLE Price (
    price_id NUMBER(10) PRIMARY KEY,
    Type VARCHAR2(20),
    price NUMBER(10)
);

-- Table to store show timings and hall/movie mapping
CREATE TABLE Shows (
    show_id NUMBER(10) PRIMARY KEY,
    show_Date DATE,
    Time VARCHAR2(10),
    Hall_id NUMBER(10) REFERENCES Halls(Hall_id),
    Movie_id NUMBER(10) REFERENCES Movies(Movie_id)
);

-- Table to store transaction/payment details
CREATE TABLE Transaction (
    Transaction_id NUMBER(10) PRIMARY KEY,
    Amount NUMBER(10),
    T_Date DATE,
    Price_id NUMBER(10) REFERENCES Price(price_id)
);

-- Table to store ticket booking details
CREATE TABLE Booked_Ticket (
    Ticket_no NUMBER(10) PRIMARY KEY,
    Transaction_id NUMBER(10) REFERENCES Transaction(Transaction_id),
    Show_id NUMBER(10) REFERENCES Shows(Show_id),
```

```
    Seat_id NUMBER(10) REFERENCES Seats(Seat_id),
    Hall_id NUMBER(10) REFERENCES Halls(Hall_id),
    Customer_id NUMBER(10) REFERENCES Customers(customer_id)
);
```

2. Inserting Data:

```
-- Inserting customer details into the Customers table
insert into customers values (1,'John doe','john.doe@gmail.com',1234567890);
insert into customers values (2,'Jain Smith','Jain.Smith@gmail.com',9574637282);
insert into customers values (3,'Michael Johnson','Michael.Johnson@gmail.com',8464632828);
insert into customers values (4,'Emily Davis','Emily.Davis@gmail.com',7836483738);
insert into customers values (5,'Robert Brown','Robert.Brown@gmail.com',6483748393);
insert into customers values (6, 'Daniel Wilson', 'Daniel.Wilson@gmail.com', 9123456789);
insert into customers values (7, 'Sophia Martinez', 'Sophia.Martinez@gmail.com', 9988776655);
insert into customers values (8, 'William Anderson', 'William.Anderson@gmail.com', 8877665544);
insert into customers values (9, 'Olivia Taylor', 'Olivia.Taylor@gmail.com', 7766554433);
insert into customers values (10, 'James Thomas', 'James.Thomas@gmail.com', 6655443322);
insert into customers values (11, 'Ava Moore', 'Ava.Moore@gmail.com', 5544332211);
insert into customers values (12, 'Benjamin Jackson', 'Benjamin.Jackson@gmail.com', 4433221100);
insert into customers values (13, 'Isabella White', 'Isabella.White@gmail.com', 3322110099);

-- Inserting movie details into the Movies table
insert into Movies values(1,'Movie_1',120,'English')
insert into Movies values(2,'Movie_2',110,'Spanish')
insert into Movies values(3,'Movie_3',130,'French')
insert into Movies values(4,'Movie_4',100,'German')
insert into Movies values(5,'Movie_5',150,'Japanese');

-- Inserting genre details into the Genre table (Movie and Genre association)
insert into Genre values(1,'Action');
insert into Genre values(2,'Drama');
insert into Genre values(3,'Comedy');
insert into Genre values(4,'Romance');
insert into Genre values(5,'Thriller');
```

```
-- Inserting hall IDs into the Halls table
insert into Halls values(1);
insert into Halls values(2);
insert into Halls values(3);
insert into Halls values(4);
insert into Halls values(5);

-- Inserting seat details into the Seats table
insert into seats values(1,100,'Standard',1,100);
insert into seats values(2,20,'Premium',1,20);
insert into Seats values(3,50,'VIP',1,50);
insert into seats values(4,30,'Balcony',1,30);
insert into seats values(5,10,'Box',1,10);
insert into seats values(6,70,'Standard',2,70);
insert into seats values(7,20,'Premium',2,20);
insert into seats values(8,15,'VIP',2,15);
insert into seats values(9,10,'Box',2,10);
insert into seats values(10,25,'Balcony',2,25);
insert into seats values(11,90,'Standard',3,90);
insert into seats values(12,40,'Premium',3,40);
insert into seats values(13,20,'VIP',3,20);
insert into seats values(14,10,'Box',3,10);
insert into seats values(15,15,'Balcony',3,15);
insert into seats values(16,60,'Standard',4,60);
insert into seats values(17,20,'Premium',4,20);
insert into seats values(18,10,'VIP',4,10);
insert into seats values(19,5,'Box',4,5);
insert into seats values(20,8,'Balcony',4,8);
insert into seats values(21,95,'Standard',5,95);
insert into seats values(22,45,'Premium',5,45);
insert into seats values(23,25,'VIP',5,25);
insert into seats values(24,15,'Box',5,15);
insert into seats values(25,10,'Balcony',5,10);

-- Inserting price details into the Price table
insert into price values(1,'Standard',20);
insert into price values(2,'Premium',15);
insert into price values(3,'VIP',20);
insert into price values(4,'Balcony',25);
insert into price values(5,'Box',30);

-- Inserting show details into the Shows table
insert into Shows values(1,to_date('2025-05-05','YYYY-MM-DD'),'18:00',1,1);
insert into Shows values(2,to_date('2025-05-05','YYYY-MM-DD'),'20:00',2,2);
insert into Shows values(3,to_date('2025-05-06','YYYY-MM-DD'),'19:00',3,3);
insert into Shows values(4,to_date('2025-05-06','YYYY-MM-DD'),'21:00',4,4);
```

```
insert into Shows values(5,to_date('2025-05-07','YYYY-MM-DD'),'17:00',5,5);

-- Inserting transaction details into the Transaction table
insert into Transaction values(1,20,to_date('2025-05-05','YYYY-MM-DD'),1);
insert into Transaction values(2,45,to_date('2025-05-05','YYYY-MM-DD'),2);
insert into Transaction values(3,60,to_date('2025-05-05','YYYY-MM-DD'),3);
insert into Transaction values(4,50,to_date('2025-05-06','YYYY-MM-DD'),4);
insert into Transaction values(5,120,to_date('2025-05-07','YYYY-MM-DD'),5);

-- Inserting booked ticket details into the Booked_Ticket table
insert into Booked_ticket values(1,1,1,1,1,1);
insert into Booked_ticket values(2,2,1,2,1,2);
insert into Booked_ticket values(3,2,1,2,1,3);
insert into Booked_ticket values(4,2,2,7,2,4);
insert into Booked_ticket values(5,3,1,3,1,5);
insert into Booked_ticket values(6,3,2,8,2,6);
insert into Booked_ticket values(7,3,3,13,3,7);
insert into Booked_ticket values(8,4,1,4,1,8);
insert into Booked_ticket values(9,4,2,10,2,9);
insert into Booked_ticket values(10,5,3,14,3,10);
insert into Booked_ticket values(11,5,1,5,1,11);
insert into Booked_ticket values(12,5,2,9,2,12);
insert into Booked_ticket values(13,5,4,19,4,13);
```

3. Displaying Data:

```
--Display Data from each table
select * from customers;
select * from Movies;
select * from Genre;
select * from price ;
select * from halls;
select * from Seats ;
select * from shows;
select * from transaction;
select * from booked_ticket;
```

4. Procedures:

- -Procedure to Book Tickets

```
CREATE OR REPLACE PROCEDURE BookTickets(
    p_customer_id IN NUMBER,
    p_show_id IN NUMBER,
    p_hall_id IN NUMBER,
    p_num_tickets IN NUMBER,
```

```
    p_seat_category IN VARCHAR2
) AS
    v_available_seats NUMBER;
    v_ticket_price NUMBER;
    v_total_price NUMBER;
    v_seat_id NUMBER;
    v_transaction_id NUMBER;
    v_ticket_no NUMBER;
BEGIN

    SELECT NVL(No_of_Seats,0)
    INTO v_available_seats
    FROM Seats
    WHERE Seat_Category = p_seat_category
      AND Hall_id = p_hall_id;

    IF v_available_seats < p_num_tickets THEN
        DBMS_OUTPUT.PUT_LINE('Not enough seats available for Hall: '
|| p_hall_id);
        RETURN;
    END IF;


    SELECT Price
    INTO v_ticket_price
    FROM Price
    WHERE Type = p_seat_category;


    v_total_price := v_ticket_price * p_num_tickets;


    SELECT NVL(MAX(Transaction_id), 0) + 1
    INTO v_transaction_id
    FROM Transaction;

    UPDATE Seats
    SET No_of_Seats = No_of_Seats - p_num_tickets
    WHERE Seat_Category = p_seat_category AND Hall_id = p_hall_id;


   INSERT INTO Transaction(Transaction_id, Amount, T_Date, Price_id)
    VALUES (v_transaction_id, v_total_price, SYSDATE,
                    (SELECT Price_id FROM Price WHERE Type =
p_seat_category));
```

```
    FOR i IN 1..p_num_tickets LOOP

        SELECT Seat_id
        INTO v_seat_id
        FROM (
            SELECT Seat_id
            FROM Seats
            WHERE Seat_Category = p_seat_category
              AND Hall_id = p_hall_id
        );


        SELECT NVL(MAX(Ticket_no), 0) + 1
        INTO v_ticket_no
        FROM Booked_Ticket;


        INSERT INTO Booked_Ticket(Ticket_no, Transaction_id, Show_id,
Seat_id, Customer_id, Hall_id)
            VALUES (v_ticket_no, v_transaction_id, p_show_id, v_seat_id,
p_customer_id, p_hall_id);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Tickets booked successfully for Hall ID:
' || p_hall_id);
    DBMS_OUTPUT.PUT_LINE('Total Amount: ' || v_total_price);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Required data not found.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' ||
SQLERRM);
END;
/
```

```
SQL> BEGIN
        BookTickets(
            p_customer_id   => 1,
            p_show_id       => 1,        ...
Show more...


Tickets booked successfully for Hall ID: 1
Total Amount: 30


PL/SQL procedure successfully completed.
```

- - Procedure for Cancellation of ticket

```
CREATE OR REPLACE PROCEDURE CancelBooking(
   p_ticket_no IN NUMBER
) AS
   v_count NUMBER;
   v_seat_category VARCHAR2(50);
   v_hall_id NUMBER;
   v_seat_id NUMBER;
BEGIN

   SELECT COUNT(*) INTO v_count
   FROM Booked_Ticket
   WHERE Ticket_no = p_ticket_no;

   IF v_count = 0 THEN
      DBMS_OUTPUT.PUT_LINE('Error: Ticket number ' || p_ticket_no ||
' does not exist.');
      RETURN;
   END IF;


   SELECT Hall_id, Seat_id INTO v_hall_id, v_seat_id
   FROM Booked_Ticket
   WHERE Ticket_no = p_ticket_no;


   SELECT Seat_Category INTO v_seat_category
   FROM Seats
   WHERE Seat_id = v_seat_id;


   DELETE FROM Booked_Ticket
   WHERE Ticket_no = p_ticket_no;


   UPDATE Seats
   SET No_of_Seats = No_of_Seats + 1
   WHERE Seat_Category = v_seat_category AND Hall_id = v_hall_id;

      DBMS_OUTPUT.PUT_LINE('Booking with Ticket Number ' ||
p_ticket_no || ' has been successfully canceled.');

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Error: Required data not found during
cancellation.');
```

```
   WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' ||
SQLERRM);
END;
/
```

```
SQL> BEGIN
         CancelBooking(p_ticket_no => 11);
     END;


Booking with Ticket Number 11 has been successfully canceled.


PL/SQL procedure successfully completed.
```

5. Functions:

– To get all bookings details

```
CREATE OR REPLACE FUNCTION GetTicketDetails
 RETURN SYS_REFCURSOR
 IS
 v_cursor SYS_REFCURSOR;
 BEGIN
 OPEN v_cursor FOR
 SELECT BT.Ticket_No, C.Customer_name, P.Type
 FROM Booked_Ticket BT
 JOIN Customers C ON BT.Customer_Id = C.Customer_id
 JOIN Transaction S ON BT.transaction_id = S.transaction_id
 JOIN Price P ON S.Price_ID = P.Price_ID;
 RETURN v_cursor;
 END;
/

DECLARE
 v_ticket_cursor SYS_REFCURSOR;
 v_ticket_id NUMBER;
 v_customer_name VARCHAR2(50);
 v_ticket_type VARCHAR2(20);
 BEGIN
 v_ticket_cursor := GetTicketDetails;
 LOOP
 FETCH v_ticket_cursor INTO v_ticket_id, v_customer_name,
 v_ticket_type;
 EXIT WHEN v_ticket_cursor%NOTFOUND;
 DBMS_OUTPUT.PUT_LINE('Ticket ID: ' || v_ticket_id || ',
 Customer Name: ' || v_customer_name || ', Ticket Type: ' ||
 v_ticket_type);
 END LOOP;
```

```
 CLOSE v_ticket_cursor;
 END;
/
```

```
Ticket ID: 4,
 Customer Name: Emily Davis, Ticket Type: Premium
Ticket ID: 6,
 Customer Name: Daniel Wilson, Ticket Type: VIP
Ticket ID: 7,
 Customer Name: Sophia Martinez, Ticket Type: VIP
Ticket ID: 8,
 Customer Name: William Anderson, Ticket Type: Balcony
Ticket ID: 9,
 Customer Name: Olivia Taylor, Ticket Type: Balcony
Ticket ID: 10,
 Customer Name: James Thomas, Ticket Type: Box
Ticket ID: 12,
 Customer Name: Benjamin Jackson, Ticket Type: Box
Ticket ID: 13,
 Customer Name: Isabella White, Ticket Type: Box


PL/SQL procedure successfully completed.
```

–To check available seats

```
CREATE OR REPLACE FUNCTION CheckAvailableSeats(
   p_show_id IN NUMBER
) RETURN NUMBER
IS
   v_hall_id        NUMBER;
   v_total_seats    NUMBER := 0;
   v_available_seats NUMBER := 0;
BEGIN

   SELECT Hall_ID INTO v_hall_id
   FROM Shows
   WHERE Show_ID = p_show_id;

   SELECT SUM(No_of_Seats) INTO v_total_seats
   FROM Seats
   WHERE Hall_ID = v_hall_id;

   v_available_seats := v_total_seats;

   RETURN v_available_seats;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No data found for Show_ID: ' ||
p_show_id);
      RETURN NULL;
   WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' ||
SQLERRM);
```

```
        RETURN NULL;
END;
/

DECLARE
 v_show_id NUMBER := 1;
 v_available_seats NUMBER;
 BEGIN
 v_available_seats := CheckAvailableSeats(v_show_id);
DBMS_OUTPUT.PUT_LINE('Available Seats for Show ' ||
 v_show_id || ': ' || v_available_seats);
 END;
/
```

```
SQL> DECLARE
       v_show_id NUMBER := 1;
       v_available_seats NUMBER;
       BEGIN  ...
Show more...


Available Seats for Show 1: 202


PL/SQL procedure successfully completed.
```

– To calculate the revenue

```
CREATE OR REPLACE FUNCTION CalculateShowRevenue
 RETURN VARCHAR2
 IS
 v_show_id NUMBER;
 v_total_revenue NUMBER := 0;
 v_show_revenue VARCHAR2(200) := '';
 CURSOR show_cursor IS
 SELECT DISTINCT Show_ID
 FROM Booked_Ticket;
 BEGIN
 FOR show_rec IN show_cursor LOOP
 v_show_id := show_rec.Show_ID;
 DECLARE
 v_show_total_revenue NUMBER := 0;
 v_ticket_price NUMBER;
 BEGIN
 FOR ticket_rec IN (SELECT * FROM Booked_Ticket
 WHERE Show_ID = v_show_id) LOOP
 SELECT Price
 INTO v_ticket_price
 FROM Price
 WHERE Price_ID = (SELECT Price_ID FROM transaction
```

```
 WHERE transaction_ID = v_show_id);
 v_show_total_revenue := v_show_total_revenue +
 v_ticket_price;
END LOOP;
 v_show_revenue := v_show_revenue || 'Show ID: ' ||
 v_show_id || ', Revenue: ' || TO_CHAR(v_show_total_revenue) ||
 CHR(10);
 v_total_revenue := v_total_revenue +
 v_show_total_revenue;
 END;
 END LOOP;
 v_show_revenue := v_show_revenue || 'Total Revenue: ' ||
 TO_CHAR(v_total_revenue);
 RETURN v_show_revenue;
 END;
/
DECLARE
 v_show_revenue VARCHAR2(200);
 BEGIN
 v_show_revenue := CalculateShowRevenue;
 DBMS_OUTPUT.PUT_LINE(v_show_revenue);
 END;
/
```

```
Show ID: 1, Revenue: 140
Show ID: 2, Revenue: 60
Show ID: 3, Revenue: 40
Show ID: 4, Revenue: 25
Total Revenue: 265
```

6. Triggers:

–To check minimum name length

```
CREATE OR REPLACE TRIGGER Phone_Number_Length_Trigger
 BEFORE INSERT OR UPDATE ON Customers
 FOR EACH ROW
 BEGIN
 IF Length(:NEW.Phone_No) < 9 THEN
 RAISE_APPLICATION_ERROR(-20019, 'Phone number cannot be
 under 10 characters!');
 END IF;
 END;
/

insert into customers values(6,'joe doe','joe.doe@gamil.com','12345');
```

```
SQL> insert into customers values(6,'joe doe','joe.doe@gamil.com','12345')

ORA-20019: Phone number cannot be
 under 10 characters!
ORA-06512: at "SQL_XK0VX94T31LEPJL7Y33V1E4IKC.PHONE_NUMBER_LENGTH_TRIGGER", line 3
ORA-04088: error during execution of trigger 'SQL_XK0VX94T31LEPJL7Y33V1E4IKC.PHONE_NUMBER_LENGTH_TRIGGER'

https://docs.oracle.com/error-help/db/ora-20019/
Error at Line: 15 Column: 0
```

– To check minimum name length

```
CREATE OR REPLACE TRIGGER Minimum_Length_Trigger
BEFORE INSERT ON Customers
FOR EACH ROW
BEGIN
IF LENGTH(:NEW.Customer_Name) < 5 THEN
RAISE_APPLICATION_ERROR(-20001, 'Customer name must be at
least 5 characters long!');
END IF;
END;
/
INSERT INTO Customers (Customer_id, Customer_Name, Email_Id,
Phone_NO)
VALUES (6, 'Trap', 'trapscam@scammer.com', '99999');
```

```
SQL> INSERT INTO Customers (Customer_id, Customer_Name, Email_Id,
      Phone_NO)
      VALUES (6, 'Trap', 'trapscam@scammer.com', '99999')

ORA-20001: Customer name must be at
 least 5 characters long!
ORA-06512: at "SQL_XK0VX94T31LEPJL7Y33V1E4IKC.MINIMUM_LENGTH_TRIGGER", line 3
ORA-04088: error during execution of trigger 'SQL_XK0VX94T31LEPJL7Y33V1E4IKC.MINIMUM_LENGTH_TRIGGER'

https://docs.oracle.com/error-help/db/ora-20001/
Error at Line: 15 Column: 0
```

VIII. Conclusion

In conclusion, our project aims to revolutionise theatre management by providing a modern, efficient, and user-friendly database management system. By replacing outdated systems with our solution, theatres can improve their operations, enhance the customer experience, and reduce costs associated with maintenance and upgrades. We've explored the critical role of a well-designed Database Management System (DBMS) in theatre management. Our project prioritised scalability, reliability, security, and integration, ensuring the system adapts to future growth and seamlessly connects with existing infrastructure. User training and ongoing refinement guarantee smooth adoption and long-term effectiveness.

References

1. Oracle Online Resources. (2024). [Online]. Available at: [https://docs.oracle.com/en/database/]

2. W3Schools SQL Tutorial. (2024). [Online]. Available at: [https://www.w3schools.com/sql/]

3. Oracle Live SQL Community. (2024). [Online]. Available at: [https://livesql.oracle.com/apex/f?p=590:1000]

4. "Database System Concepts" by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. Published by McGraw-Hill Education.

5. MySQL Documentation. (2024). [Online]. Available at: [https://dev.mysql.com/doc/]

6. "Oracle Database 19c Documentation" by Oracle Corporation. [Online]. Available at: [https://docs.oracle.com/en/database/oracle/oracle-database/19/]